

University of Stuttgart
Germany

Advanced Laboratory Course

Physics Informed Neural Networks

Jakob Wallner

Supervisor: Tobias Fellner

Version: 0

Contents

1	Theory	2
1.1	Neural Networks	2
1.2	Training a Neural Network	3
1.3	Physics-Informed Neural Networks	3
2	Day 1	3
3	Day 2	3
4	Conclusion	3
5	References	3

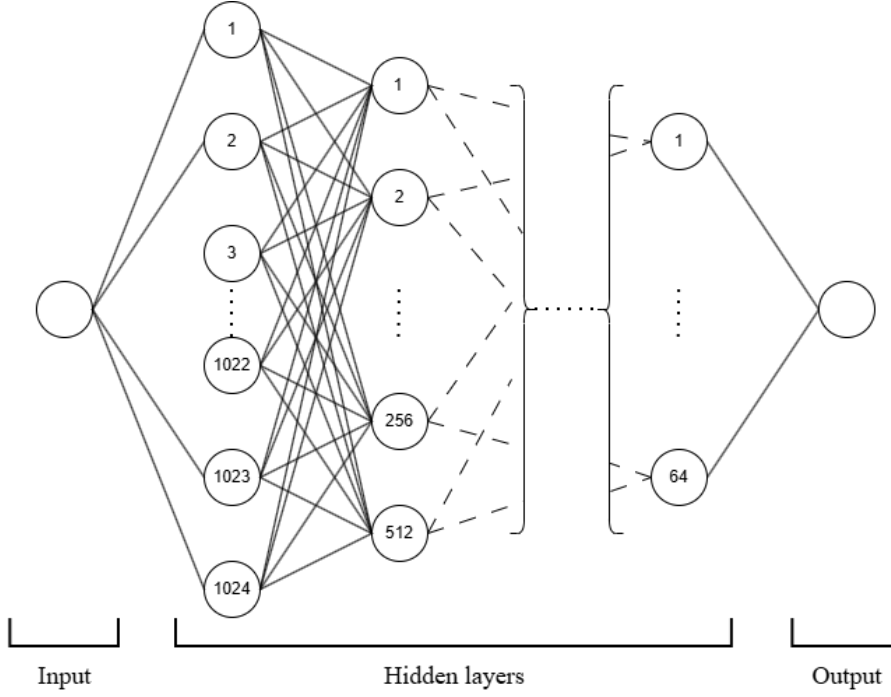


Figure 1: Schematic representation of a simple feedforward neural network with several hidden layers.

1 Theory

1.1 Neural Networks

The modern Neural Networks (NNs) are based on a mathematical model of the neuron, a so-called artificial neuron or perceptron. This neuron receives multiple inputs $\mathbf{x}$ and applies a learnable weight w_i to each input, sums them up and adds a bias b to produce an output y . This information is then passed through a non-linear activation function f to introduce non-linearity into the model, allowing it to learn complex patterns in the data.

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right) = f(\mathbf{x} \cdot \mathbf{w} + b) \quad (1.1)$$

A common activation function is the Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$. In order to advance in high-dimensional problems, multiple layers of neurons are stacked together to form a deep neural network. The arrangement of these layers, called architecture, can vary widely and changes the way the network learns and processes information. Usual architectures include feedforward networks, convolutional neural networks (CNNs) and recurrent neural networks (RNNs).

A better way to represent the architecture of a neural network is through matrix operations. A hidden layer l with m neurons receives an input vector $\mathbf{a}^{(l-1)}$ of size n from the previous layer, applies

a weight matrix $\mathbf{W}^{(l)}$ of size $m \times n$ and a bias vector $\mathbf{b}^{(l)}$ of size m , and produces an output vector $\mathbf{a}^{(l)}$ of size m .

$$\mathbf{a}^{(l)} = f\left(\mathbf{W}^{(l)} \cdot \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right) \quad (1.2)$$

Stacking multiple layers allows the network to become a highly complex and non-linear function. The combination of the non-linear activation functions and the learnable parameters (weights and biases) enables the network to approximate a wide variety of functions, making it a powerful tool for tasks such as classification, regression, and generative modeling.

1.2 Training a Neural Network

In order to improve the performance of a NN, it needs to be trained on a dataset. This process is realized by determining the optimal network parameters (weights $\mathbf{W}$ and biases $\mathbf{b}$) that minimize a loss function $\mathcal{L}$, which quantifies the difference between the predicted output and the true output. A simple example of a loss function is the mean squared error (MSE) for regression tasks:

$$\mathcal{L}_{MSE}(\hat{y}, y) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \quad (1.3)$$

where $\hat{y}$ is the predicted output, y is the true output, and N is the number of samples in the dataset.

1.3 Physics-Informed Neural Networks

2 Day 1

3 Day 2

4 Conclusion

5 References

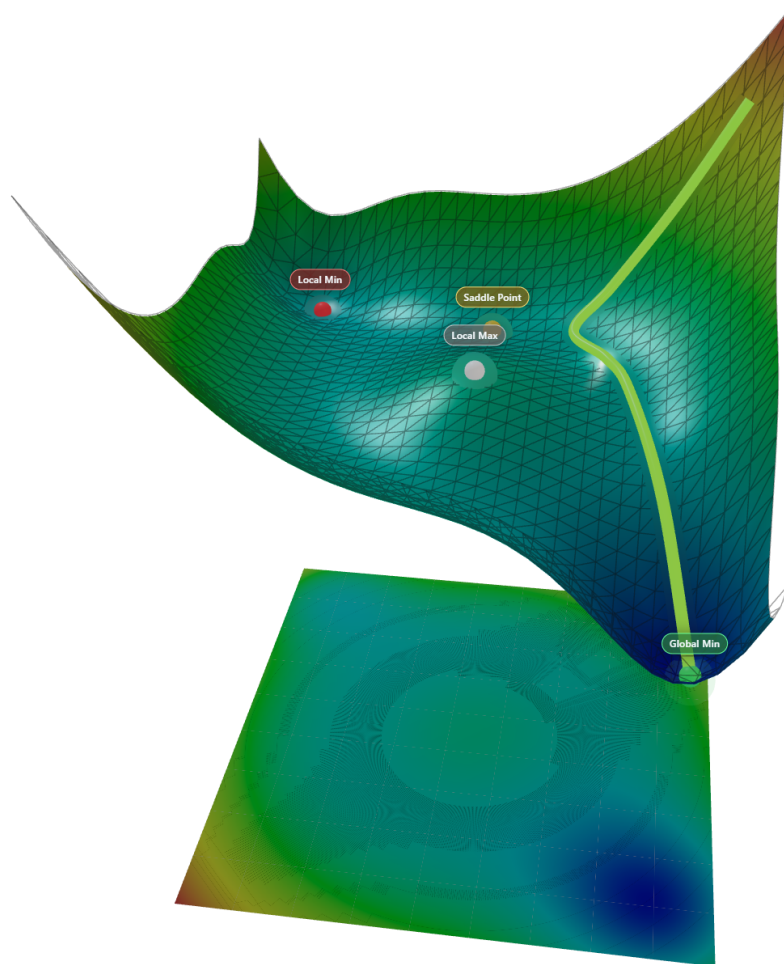


Figure 2: Schematic representation of the training process of a neural network, where the loss function is minimized by adjusting the weights and biases.